

Advance Operating System

Term Evaluation (Odd) Semester Examination - September 2025

Question Paper with Model Answers

Course	MCA	Semester	I
Paper	Advance Operating System	Paper Code	TMC-104 / TMD-105
Time	1.5 hours	Maximum Marks	50
Pattern	5 questions x 10 marks; attempt one sub-question (a or b) of each	Main reference	Silberschatz, Galvin, Gagne - Operating System Concepts, 9th Ed.

How to use this document

- Each question from the paper is reproduced (process tables are written inline). **Both options (a) and (b) are answered**, so that the whole syllabus is covered during revision.
- In the exam you need to attempt only one option per question. Each answer is written to carry **10 marks**: definition, diagram/table, points, example and conclusion.
- Numerical questions (Q2 (b), Q4 (b)) show every step: Gantt chart, formula, table and average, plus a verification check. A few extra measures are added for revision.
- The **Exam tip** boxes at the end of each answer list what must not be missed. The last pages contain a one-page revision sheet and practice questions.

Contents

Question 1	2
Question 2	9
Question 3	13
Question 4	18
Question 5	23
Quick Revision Sheet	28

Question	Topics covered
Q1	Kernel vs shell; files, file systems, file organization, access methods, allocation methods
Q2	Domain of protection, access matrix, access control; FCFS scheduling numerical
Q3	Single-level and multi-level directories, fragmentation and disk management; starvation in SJF and aging
Q4	SVR3 (traditional UNIX) scheduling; Round Robin numerical with Gantt chart, WT, TAT, RT
Q5	Effect of CPU scheduling on performance; comparison of RR with FCFS and SJF

Question 1

10 Marks | Attempt any ONE sub-question in the exam | CO1 | Both options are answered below for revision

Q1 (a) [10 Marks, CO1]

Differentiate between kernel and shell. Explain their roles in operating system design.

1. Introduction

An operating system (OS) is system software that manages computer hardware, provides a base for application programs and acts as an intermediary between the user and the hardware. Internally, an OS is usually divided into two cooperating parts: the **kernel**, which does the real work of managing resources, and the **shell**, which is the interface through which the user talks to the kernel. Silberschatz (Ch. 2) describes these as the *system-call interface* (kernel side) and the *command interpreter* (user-interface side) of the operating system.

2. Kernel

The **kernel** is the central, most privileged component of the OS. It is loaded into main memory at boot time and stays resident until shutdown. It runs in **kernel (supervisor) mode**, so it can execute privileged instructions and directly access hardware. Silberschatz calls it 'the one program running at all times on the computer'.

Main functions of the kernel

- **Process management** - creation, termination, scheduling, context switching, synchronisation and inter-process communication (IPC).
- **Memory management** - allocation and de-allocation, paging/segmentation, virtual memory, protection between processes.
- **File-system management** - files, directories, allocation, free-space management, access permissions.
- **Device management** - device drivers, buffering, caching, spooling, interrupt handling.
- **System-call interface** - the controlled gateway (through traps) by which user programs request kernel services such as `open()`, `read()`, `fork()`.
- **Protection and security** - dual-mode operation, memory protection, CPU timer, access control.

Types of kernel structure (Silberschatz Sec. 2.7)

Kernel type	Idea	Advantage / Disadvantage	Example
Monolithic	All OS services run in one large program in kernel space.	Fast (no message passing) / hard to maintain, one bug can crash the system.	Traditional UNIX, Linux
Layered	OS is split into layers; layer N uses only services of layer N-1.	Easy debugging and verification / careful layer design and some overhead.	THE system
Microkernel	Only minimal services (IPC, basic scheduling, memory) in kernel; others run as user-space servers.	Secure, extensible, portable / message-passing overhead.	Mach, QNX
Modular (LKM)	Core kernel plus dynamically loadable modules.	Flexible, efficient / less strict than layering.	Linux, Solaris
Hybrid	Combination of monolithic speed and microkernel structure.	Practical compromise / complex design.	Windows NT, macOS (XNU)

3. Shell

The **shell** is a **command interpreter** - a user-level program that reads commands typed (or clicked) by the user, interprets them and asks the kernel to carry them out through system calls. It is not part of the kernel; it runs in **user mode** like any other application. It gets its name because it forms an outer 'shell' around the kernel. Silberschatz (Sec. 2.2.1) notes that there are two ways a command interpreter can be built: (i) the interpreter itself contains the code for every command, or (ii) commands are separate system programs which the interpreter simply loads and runs (the UNIX approach - which lets new commands be added without changing the shell).

Main functions of the shell

- **Command interpretation** - reads a line, parses it into command name and arguments.
- **Program execution** - creates a child with `fork()`, loads the program with `exec()`, and `wait()`s for it (or runs it in the background with `&`).
- **I/O redirection and pipes** - `<`, `>`, `|` connect commands.
- **Wild-card expansion, variables and environment** - `*`, `c`, `$PATH`, aliases, history.
- **Scripting** - variables, loops, conditionals and functions allow automation of repetitive work.
- **Job control** - foreground/background jobs, `j` jobs, `fg`, `bg`, signals such as `Ctrl+C`.

Types of shell: CLI shells - Bourne shell (`sh`), Bash, C shell (`csh`), Korn shell (`ksh`), Zsh, Windows PowerShell/`cmd`; GUI shells - Windows Explorer, GNOME, KDE, macOS Aqua; touch-screen shells on mobile devices.

Layered view: where kernel and shell sit

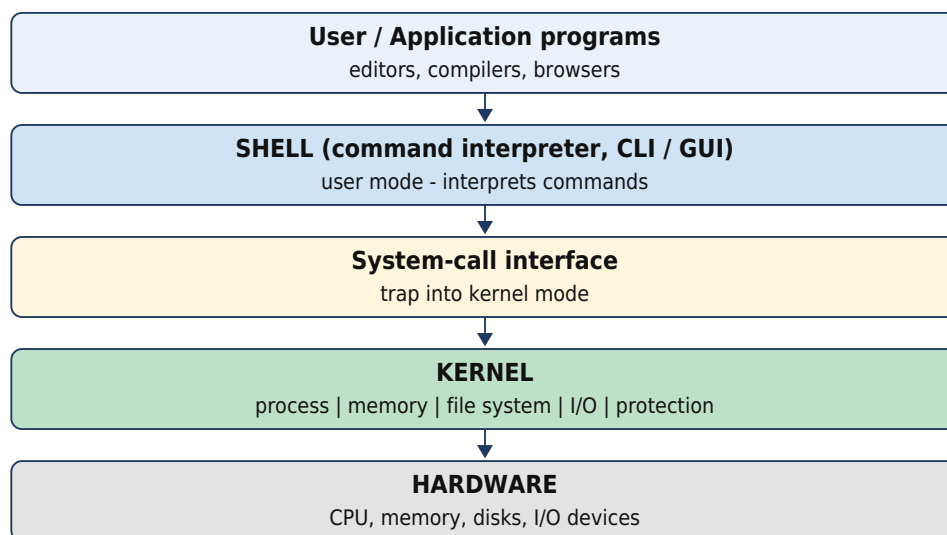


Fig. 1 - Position of the shell and kernel in operating-system design

4. Difference between Kernel and Shell

Basis	Kernel	Shell
Definition	Core of the OS that manages all hardware and system resources.	Command interpreter that lets the user communicate with the kernel.
Position	Innermost layer, directly above the hardware.	Outer layer between the user and the kernel.
Nature	Mandatory part of every OS; the OS cannot run without it.	Ordinary user-level program; system can run without a particular shell.
Execution mode	Kernel (privileged / supervisor) mode.	User mode (non-privileged).
Hardware access	Interacts directly with CPU, memory and devices.	Has no direct hardware access; uses kernel through system calls.
Interaction with user	No direct interaction with the user.	Direct interaction through commands or graphical actions.
Main job	Resource management, scheduling, memory, file and device management.	Read, parse and launch commands, redirection, pipes, scripting.
Language of interface	System calls (C/assembly, machine-oriented).	Command language (Bash, csh, PowerShell) - human-oriented.
Memory residence	Permanently loaded in main memory.	Loaded when a user logs in / opens a terminal; ends with the session.
Replaceability	Replacing it means replacing the OS itself.	Easily replaced: a user can switch from bash to zsh with one command.
Effect of a crash	Kernel panic / system crash (blue screen).	Only that shell session ends; system stays up.
Examples	Linux kernel, Windows NT kernel, XNU, Mach.	bash, sh, csh, ksh, zsh, PowerShell, GNOME.

5. Roles of kernel and shell in operating-system design

- Abstraction and resource management** - the kernel hides hardware details and presents clean abstractions (process, file, virtual memory, socket) and shares the resources fairly among competing programs.
- Protection and security through dual mode** - only the kernel executes privileged instructions. The shell (and all user programs) must request services through system calls, so a faulty user program cannot damage the system.
- Separation of mechanism and policy** - the kernel provides mechanisms (scheduling, page tables); policies (which commands, what priority) are chosen at a higher level. This principle keeps the design flexible.
- User-friendliness** - the shell converts human-readable commands into system calls; different user categories (novice, expert, administrator) can be served by different shells (GUI vs CLI vs scripts).
- Modularity and extensibility** - since the shell is outside the kernel, the interface can be redesigned without touching kernel code, and kernel features can change without the user noticing (portability of the interface).
- Automation** - shell scripts combine many system calls and utilities to automate tasks such as backups, installation and log analysis.
- Design goals achieved** - efficiency and reliability (small trusted kernel), convenience (shell), and portability (standard interfaces such as POSIX).

Example: what happens when the user types `ls | wc -l`

- The shell reads the line, parses it and recognises a pipe between two commands.
- It calls `pipe()` to ask the kernel for a pipe, then `fork()` twice to create two child processes.
- Each child uses `dup2()` to attach its standard output/input to the pipe and then `exec()`s `ls` and `wc`.
- The kernel schedules both processes, performs the file-system reads for `ls`, passes data through the pipe buffer and finally writes the count to the terminal driver.

5. The shell `wait()` for the children to terminate and prints the prompt again.

Skeleton of a simple shell loop

```
while (1) {
    printf("myshell> ");
    read_line(cmd);           // 1. read the command typed by the user
    parse(cmd, args);         // 2. split into program name and arguments
    pid = fork();             // 3. system call: create a child process
    if (pid == 0) {
        execvp(args[0], args); // child: load and run the requested program
        perror("exec failed");  // reached only if exec fails
    } else {
        waitpid(pid, &status, 0); // 4. parent (shell) waits for the child
    }
}
```

Exam tip - key points to remember

- Kernel = privileged, always in memory, manages resources. Shell = user-mode interpreter, interface to the kernel.
- Shell requests every service from the kernel using system calls (fork, exec, open, read, write, pipe).
- Structures of kernel: monolithic, layered, microkernel, modular, hybrid - write at least two with examples.

Q1 (b) [10 Marks, CO1]

Explain files and file systems. Discuss different types of file organization and access methods.

1. File

A **file** is a named collection of related information that is recorded on secondary storage. From the user's point of view it is the smallest allotment of logical secondary storage - data cannot be written to secondary storage unless it is inside a file. Files may hold programs (source and object) or data (numeric, text, binary). The OS maps each file onto physical devices such as magnetic disks or SSDs, so that the user sees a uniform logical view (Silberschatz, Ch. 11).

File attributes

Attribute	Meaning
Name	Symbolic, human-readable name (the only attribute kept in human-readable form).
Identifier	Unique number (e.g. inode number) by which the file system identifies the file.
Type	Needed in systems supporting different file types (text, executable, image ...).
Location	Pointer to the device and to the location of the file on that device.
Size	Current size in bytes/words/blocks and possibly the maximum allowed size.
Protection	Access-control information: who can read, write, execute.
Time, date, user id	Creation, last modification and last use - useful for protection, security and usage monitoring.

File operations

- **Create** - find space in the file system and add an entry in the directory.
- **Write / Read** - system call names the file and the data; the system keeps a *current-file-position pointer* per process.
- **Reposition (seek)** - move the current-file-position pointer without doing any I/O.
- **Delete** - release the file space and remove the directory entry. **Truncate** - erase the contents but keep the attributes.

- **Open / Close** - `open(F)` searches the directory and copies the entry into the open-file table; `close(F)` removes it.

Open-file tables: the *per-process* table holds the current pointer and access rights; the *system-wide* table holds the file-open count, disk location and dates. When the count becomes zero the entry is removed. File **locks** (shared/exclusive, mandatory/advisory) coordinate access by several processes.

File types are often indicated by an extension (.c, .exe, .txt, .jpg, .zip). UNIX stores a 'magic number' at the beginning of a file instead.

2. File system

The **file system** is the part of the OS that provides the mechanism for on-line storage of and access to both data and programs. It consists of two distinct parts: a **collection of files** (each storing related data) and a **directory structure** which organises and provides information about all the files in the system. Disk-based file systems are designed to satisfy two goals: (i) to allow data to be stored, located and retrieved easily; (ii) to give efficient and convenient access to the disk.

Layered file-system structure (Silberschatz Sec. 12.1)

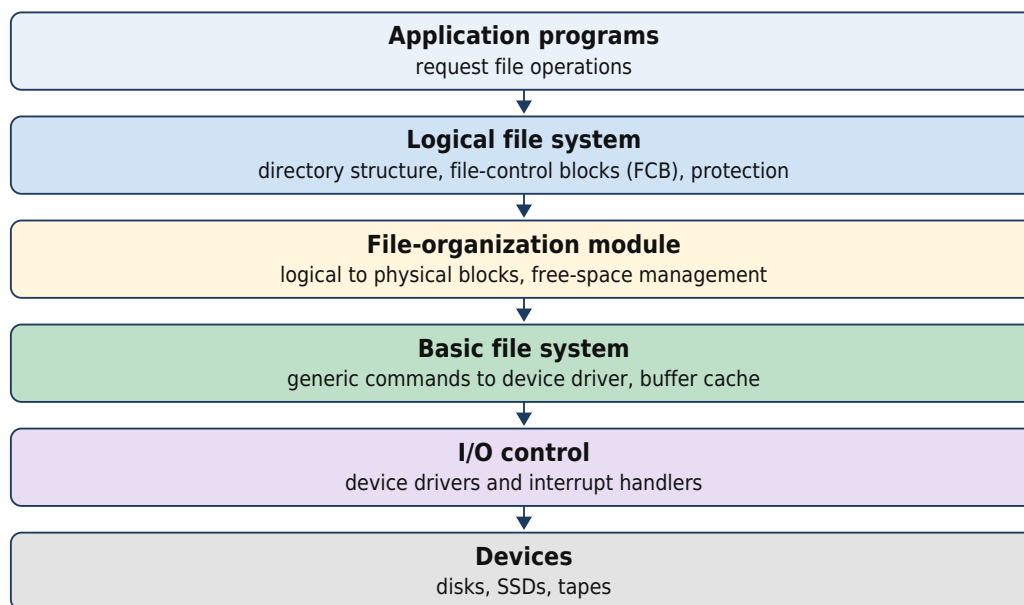


Fig. 2 - Layered file system

On-disk and in-memory structures

Structure	Purpose
Boot control block	Information needed to boot an OS from that volume (first block of the volume).
Volume control block (superblock)	Number of blocks, block size, free-block count and pointers, free-FCB count and pointers.
Directory structure	Organises the files; maps file names to FCBs/inode numbers.
Per-file FCB (inode)	Permissions, owner, size, dates and pointers to the data blocks.
Mount table, directory cache	In memory: information about mounted volumes; recently accessed directories.
System-wide and per-process open-file tables	In memory: track opened files, their FCB copy, file pointer and access mode.

Allocation methods (how disk blocks are given to files)

Method	How it works	Advantages	Disadvantages
Contiguous	Each file occupies a set of consecutive blocks; directory stores start block and length.	Simple; fast sequential and direct access; minimal seeks.	External fragmentation; file growth is difficult; size must be known.
Linked	Each block contains a pointer to the next block; directory stores first (and last) block. FAT keeps the links in a table.	No external fragmentation; files can grow easily.	No efficient direct access; pointer overhead; a lost pointer breaks the file.
Indexed	An index block holds the addresses of all data blocks of the file (UNIX inode: direct + single/double/triple indirect).	Supports direct access; no external fragmentation.	Index block overhead; large files need multi-level index.

Free-space management: *bit vector* (one bit per block), *linked list* of free blocks, *grouping* (first free block holds addresses of n free blocks), *counting* (address + number of contiguous free blocks) and *space maps* (ZFS).

Well-known file systems: FAT32, NTFS, exFAT (Windows); ext3/ext4, XFS, Btrfs (Linux); APFS (macOS); ISO 9660 (CD-ROM); NFS (network).

3. File organization

File organization refers to the logical structuring of the records of a file - the way records are arranged and related to each other inside the file. The choice affects speed of retrieval, ease of update, storage efficiency and reliability.

Organization	Description	Best suited for	Limitations
Pile (heap)	Records are stored in the order they arrive; no structure; each record self-describing.	Data collected for later processing; unpredictable data.	Searching is exhaustive (linear scan).
Sequential file	Fixed-format records stored in order of a key field; records processed one after another.	Batch processing (payroll, billing) where most records are used.	Slow random access; insertion needs a transaction file and merge.
Indexed sequential (ISAM)	Sequential file plus an index on the key and an overflow area; index gives fast jump then sequential scan.	Both sequential and random access are needed (banking, inventory).	Extra index storage; overflow area needs periodic reorganisation.
Indexed file	Records accessed only through one or more indexes (multiple keys possible).	Applications requiring fast access by different fields.	Index maintenance cost on insert/delete.
Direct / hashed file	A hash function on the key gives the block address directly; no ordering.	Very fast random access, e.g. lookup tables, timetables.	Collisions; no sequential processing; wastes space.

4. File-access methods

Access methods define how the information in a file is read and written by programs (Silberschatz Sec. 11.2).

(i) Sequential access

The simplest and most common method. Information is processed in order, one record after another. A `read next` reads the next portion and advances the file pointer automatically; `write next` appends to the end of the file; `reset` moves back to the beginning. Some systems can skip forward or backward n records. It is based on the tape model but works equally well on random-access devices. Examples: compilers, text editors, log files.

(ii) Direct (relative) access

The file is viewed as a numbered sequence of fixed-length blocks or records, and any block can be read or written in any order: read n , write n , position to n , where n is a **relative block number** (0 = first block of the file). It is based on the disk model and is ideal for databases, airline reservation systems and other large data sets needing immediate response. Sequential access can be simulated on direct access by keeping a counter cp :
 read next = read cp ; $cp = cp + 1$.

(iii) Indexed (other) access methods

An **index**, like the index of a book, contains pointers to the blocks. To find a record, the index is searched first (binary search if it is sorted) and the pointer is used to access the file directly. For very large files the index itself becomes large, so a **multi-level index** is created (an index to the index) - the idea used by IBM's ISAM. Keeping the primary index in memory reduces disk I/O.

Basis	Sequential access	Direct access	Indexed access
Order of access	Strictly in order, record by record.	Any block, in any order.	Through index, then to the record.
Speed for random request	Slow (must pass earlier records).	Fast.	Fast; one extra index lookup.
Operations	read next, write next, reset.	read n , write n , position to n .	search index, then read/write record.
Overhead	None.	Needs fixed-size records/blocks.	Index storage and its maintenance.
Typical use	Compilers, editors, backups.	Databases, reservation systems.	Large databases, ISAM/B-tree files.

5. Directory operations and protection (brief)

The directory supports search, create, delete, list, rename and traverse operations on files (structures are described in Q3 (a)). The file system also protects files against improper access by controlling read, write, execute, append, delete and list operations through access-control lists or the UNIX owner/group/others permission bits (see Q2 (a)).

Exam tip - key points to remember

- File = named collection of related data on secondary storage; file system = files + directory structure + management code.
- Six layers of a file system: application, logical FS, file-organization module, basic FS, I/O control, devices.
- Organization: pile, sequential, indexed-sequential, indexed, direct/hashed. Access: sequential, direct, indexed.

Question 2

10 Marks | Attempt any ONE sub-question in the exam | CO2 | Both options are answered below for revision

Q2 (a) [10 Marks, CO2]

Write short notes on the following: (a) Domain of Protection (b) Access Control

A. Domain of Protection

Protection - the background (Silberschatz, Ch. 14)

Protection refers to a mechanism that controls the access of programs, processes or users to the resources defined by a computer system. It is an *internal* problem (controlling access inside one system), whereas security is an *external* problem (defending the system against outside threats). Goals of protection: to prevent mischievous violation of an access restriction, and to ensure that each program component uses system resources only in ways consistent with stated policies (i.e. improve reliability by detecting interface errors early).

Principle of least privilege: programs, users and systems should be given *just enough privileges to perform their tasks*. Example: a mail-reading program needs read access to the mailbox only and should not be able to modify system files. **Mechanism vs policy:** the OS provides mechanisms (how); policy (what will be done) is decided by users or administrators, so policies can change without changing the mechanism.

Domain of protection

A computer system is a collection of **processes** and **objects**. Objects are *hardware* objects (CPU, memory segments, printers, disks) and *software* objects (files, programs, semaphores). Each object has a unique name and a set of operations that can be performed on it (e.g. read/write on a file, wait/signal on a semaphore). A process should be allowed to access only those resources it is currently authorised to access and only those it requires to complete its task (**need-to-know principle**).

A **domain** is the set of objects together with the operations allowed on each of them. Each element of a domain is an **access right**, written as an ordered pair **<object-name, rights-set>**. For example, domain D with right <file F, {read, write}> means a process executing in D can read and write file F but cannot execute or delete it. Domains may **overlap** - they can share access rights (e.g. D1 and D3 may both hold <O4, {print}>).

Association between a process and a domain may be:

- **Static** - the set of resources available to the process is fixed for its whole lifetime. This is simple but may violate need-to-know, because the domain must contain the rights for all phases of the process.
- **Dynamic** - the process can *switch* from one domain to another (*domain switching*), so that at any moment it has only the rights it needs. A mechanism is required to change the contents of the domain, or to let the process switch domain.

Realisation of a domain

Domain = ...	Explanation
User	Objects accessible depend on the identity of the user. Domain switch happens when the user changes (log out and log in).
Process	Access depends on the identity of the process. Domain switch happens when one process sends a message to another and waits for a reply.
Procedure	Access depends on the local variables of the procedure. Domain switch occurs on a procedure call.

Example 1: UNIX domains

In UNIX a domain is associated with the **user**. Each file has an *owner-id* and a *domain bit*. Domain switching is implemented through the file system: the domain bit is the **setuid** bit (with a similar setgid bit for groups). When a user runs a file whose setuid bit is on, the effective user-id of the process becomes the **owner** of the file instead of the user who started it. The domain of the process therefore switches for the duration of the program (e.g. the `passwd` command runs as root so that an ordinary user can change his own entry in the password file). Because it enlarges privileges, setuid programs must be written very carefully.

Example 2: MULTICS rings

MULTICS organises domains as **concentric rings** numbered 0 to 7. Ring 0 (innermost) has the most privileges; if $j < i$ then D_i is a subset of D_j - ring D_j has every privilege of ring D_i . A process running in ring i cannot access a segment of a more privileged ring without a controlled call through a *gate* (call to a procedure in an inner ring). The ring structure is a generalisation of the dual-mode (user/kernel) design and is used, for example, in Intel x86 privilege levels 0-3.

Ring 0 = kernel (most privileged) ... Ring 7 = least privileged user code

The access matrix (model of protection)

The general model of protection is the **access matrix**: **rows represent domains**, **columns represent objects**, and entry $access(i, j)$ is the set of operations that a process executing in domain D_i may invoke on object O_j . Domain switching is included by treating domains as objects with a *switch* right.

Domain \ Object	F1	F2	F3	Printer	D2 (switch)
D1	read		read		switch
D2				print	
D3		read	execute		
D4	read, write		read, write		

Special rights allow the matrix to change in a controlled way: **copy** (marked *), **owner** (add/remove any right in a column) and **control** (modify the row of another domain). The matrix is normally sparse, so it is stored as a **global table**, as **access lists** per object (columns), as **capability lists** per domain (rows), or with a **lock-key** scheme.

B. Access Control

Access control is the mechanism by which the OS decides *who* (user, process, group) may perform *which* operation (read, write, execute, append, delete, list) on *which* object (file, directory, device, memory). It enforces the protection policy after the user has been identified by **authentication** (passwords, biometrics, tokens). Access control therefore answers the question 'What is this authenticated subject allowed to do?' (Silberschatz Sec. 11.6 and 14.6).

Types of access on a file

Read, Write, Execute, Append, Delete, List (attributes/name listing). Higher-level functions such as copy and rename can be built from these.

1. Access-Control List (ACL)

The most general approach is to associate with every file and directory an **identity-dependent list** stating the user names and the types of access allowed for each. When a user requests access, the OS examines the ACL of the object; if the user is listed for that access, it is granted, else a protection violation occurs. *Advantages*: fine-grained, easy to revoke access (edit the list). *Disadvantages*: constructing the list is tedious when users are not known in advance, and the directory entry becomes variable-sized, complicating space management.

2. Condensed / UNIX permission bits

To shorten the list, users are classified into three classes: **Owner** (user who created the file), **Group** (set of users sharing the file with similar access) and **Universe / Others** (all other users). With three access bits (r, w, x) for each class, only **9 bits** are needed. Example: `-rwxr-x-x` or octal 751 means owner has rwx (7), group has r-x (5) and others have only execute (1). The `chmod`, `chown` and `chgrp` commands change these bits.

3. Capability lists

A **capability** is a protected pointer/token naming an object and the rights held for it. Each domain (process) keeps a list of capabilities and can access an object only if it holds the right capability. Capabilities are protected by the OS (kept in kernel space) and cannot be forged. They are efficient for the running process but revocation is harder.

4. Role-Based Access Control (RBAC) and mandatory access control

Solaris 10 and later versions use **role-based access control**: privileges are assigned to *roles*, and roles are assigned to users. A user can assume a role only with the proper password, so the principle of least privilege is applied without an all-powerful root. Other models: **DAC** (discretionary - the owner decides), **MAC** (mandatory - system-wide policy based on security labels, e.g. SELinux) and **RBAC**.

Comparison: Access list vs Capability list

Basis	Access-control list (ACL)	Capability list
Stored with	The object (column of the access matrix).	The domain / process (row of the access matrix).
Question answered	Who can access this object?	What can this domain access?
Revocation of rights	Easy: edit the list of the object.	Difficult: capabilities are spread over many domains.
Checking cost at access	Search of the list on every access (can be slow).	Fast: presenting the capability is enough.
Best for	File systems (UNIX, Windows NTFS).	Distributed/capability-based systems.

Revocation of access rights

In a dynamic protection system it may be necessary to revoke rights: immediate or delayed, selective or general, partial or total, temporary or permanent. With ACLs revocation is straightforward. With capabilities it is done by re-acquisition, back-pointers, indirection through a global table entry, or by keys.

Exam tip - key points to remember

- Domain = set of <object, rights> pairs; static or dynamic binding with a process; realised as user, process or procedure.
- UNIX: setuid bit for domain switching. MULTICS: rings 0-7, inner ring most privileged.
- Access matrix: rows = domains, columns = objects; implemented as ACL (columns) or capability lists (rows).
- Access control in files: ACL, owner-group-universe (rwx = 9 bits), RBAC; enforced after authentication.

Q2 (b) [10 Marks, CO2]

Consider the following processes: P1 (AT 0, BT 8), P2 (AT 1, BT 4), P3 (AT 2, BT 9), P4 (AT 3, BT 5). Using the FCFS algorithm prepare the Gantt chart. Calculate Turnaround Time (TAT), Waiting Time (WT) and their averages.

1. First-Come, First-Served (FCFS) scheduling - concept

FCFS is the simplest CPU-scheduling algorithm: the process that requests the CPU first is allocated the CPU first. It is implemented with a **FIFO queue**. When a process enters the ready queue its PCB is linked to the tail; when the CPU is free it is allocated to the process at the head of the queue. FCFS is **non-preemptive** - once the CPU has been given to a process, it keeps it until it terminates or requests I/O.

Formulae used (Silberschatz Sec. 5.2)

- **Completion Time (CT)** = time at which the process finishes execution.
- **Turnaround Time (TAT)** = CT - Arrival Time (AT) (sum of waiting in ready queue, executing and doing I/O).
- **Waiting Time (WT)** = TAT - Burst Time (BT) (time spent waiting in the ready queue).
- **Average TAT** = (sum of all TAT) / n, **Average WT** = (sum of all WT) / n.

2. Given data

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

3. Order of execution and Gantt chart

Processes are served strictly in order of arrival: P1 (arrives at 0), then P2 (1), P3 (2) and P4 (3). There is no idle time because P1 arrives at time 0 and every later process has already arrived before the CPU becomes free.



Fig. 3 - Gantt chart for FCFS (time in units)

Start/finish: P1: 0 to 8 | P2: 8 to 12 ($8 + 4$) | P3: 12 to 21 ($12 + 9$) | P4: 21 to 26 ($21 + 5$).

4. Calculation of CT, TAT and WT

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$
P2	1	4	12	$12 - 1 = 11$	$11 - 4 = 7$
P3	2	9	21	$21 - 2 = 19$	$19 - 9 = 10$
P4	3	5	26	$26 - 3 = 23$	$23 - 5 = 18$
Total		26		61	35

Average Turnaround Time = $(8 + 11 + 19 + 23) / 4 = 61 / 4 = 15.25$ time units

Average Waiting Time = $(0 + 7 + 10 + 18) / 4 = 35 / 4 = 8.75$ time units

5. Additional performance measures

- **CPU utilisation** = $26 / 26 = 100\%$ (no idle time).
- **Throughput** = $4 \text{ processes} / 26 \text{ units} = 0.154$ processes per time unit.
- **Average response time** = average of (first CPU - AT) = $(0 + 7 + 10 + 18) / 4 = 8.75$, the same as WT because FCFS is non-preemptive.

6. Observations on FCFS

- P4 needs only 5 units but waits 18 units, because it is stuck behind the long process P3 - the **convoy effect** (many short processes wait for one long process), which lowers CPU and device utilisation.
- The average waiting time is **often long** and depends heavily on arrival order; if the shortest jobs had run first the average would be smaller (see Q5 for SJF/RR comparison).
- FCFS is easy to write and understand, and it is fair in the sense of arrival order and free from starvation. But it is unsuitable for interactive/time-sharing systems because it is non-preemptive.

Verification

Sum of burst times = $8 + 4 + 9 + 5 = 26$ = finishing time of P4. Also, total WT = total TAT - total BT = $61 - 26 = 35$
✓

Question 3

10 Marks | Attempt any ONE sub-question in the exam | CO2 / CO1 | Both options are answered below for revision

Q3 (a) [10 Marks, CO2]

Write short notes on the following: (i) Single-level and multi-level directories (ii) Disk fragmentation and management techniques

(i) Single-level and Multi-level Directories

A **directory** is a symbol table that translates file names into their directory entries (file-control blocks). It organises the files of a volume. Directory operations are: **search** for a file, **create** a file, **delete** a file, **list** a directory, **rename** a file and **traverse** the file system. The main goals of a directory structure are efficiency (locate a file quickly), naming (convenient for users - two users may use the same name, one file may have several names) and grouping (logical grouping of files by property, e.g. all Java programs). (Silberschatz Sec. 11.3)

1. Single-level directory

The simplest structure: **all files are contained in the same directory**, which is easy to support and understand.

Directory :	[cat]	[bo]	[a]	[test]	[data]	[mail]	[cont]	[hex]
Files :	file	file	file	file	file	file	file	file

Limitations: (a) *naming problem* - all files must have unique names; with many users the chance of name clash is high; (b) *grouping problem* - no way to group files logically; (c) the directory becomes very long as the number of files grows, so searching and remembering names is difficult; (d) no protection between users. It was used in early microcomputers and is suitable only for single-user systems with few files.

2. Multi-level directory

To overcome the limitations, the directory is organised into **several levels**. The important multi-level forms are:

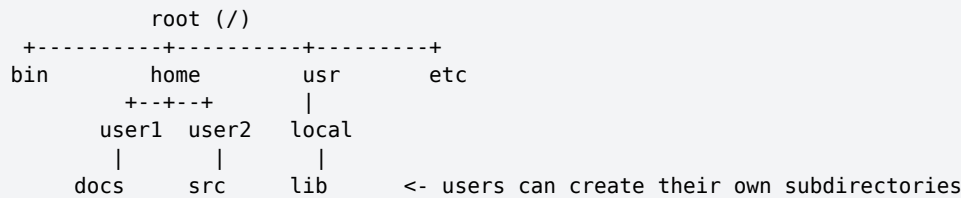
(a) Two-level directory. Each user has his own **User File Directory (UFD)**; a system-wide **Master File Directory (MFD)** indexed by user name points to the UFDs. When a user logs in the MFD is searched and the user's UFD is used for all later file operations.

MFD (master file directory)		
+-----+ +-----+ +-----+		
user 1	user 2	user 3
+-----+ +-----+ +-----+		
UFD of 1	UFD of 2	UFD of 3
[cat][bo][a]	[test][a][x]	[data][a][hex]

Path name = user name + file name, e.g. /user2/test

Advantages: different users may use the same file name; solves naming problem; provides isolation (protection) between users; efficient searching. **Disadvantages:** users are isolated - cooperation and file sharing is difficult; still no grouping inside a user's own area. System files are accessed via a special user or a *search path*.

(b) Tree-structured (hierarchical) directory. The two-level structure generalised to a tree of arbitrary height. The tree has a **root**; every file has a unique **path name**. A directory contains files or subdirectories (a bit in the entry distinguishes them). Each process has a **current directory**; paths may be *absolute* (from root, /usr/bin/ls) or *relative* (from the current directory, ../docs/a.txt). System calls such as mkdir, rmdir and cd manage directories. Deleting a non-empty directory requires either refusing or recursive deletion (rm -r).



(c) Acyclic-graph directory. Allows directories/files to be **shared** by several users through *links* (UNIX hard links and symbolic links). It is more flexible but complex: a file may have multiple absolute names (aliasing), deletion needs a reference count to avoid *dangling pointers*. **(d) General-graph directory** permits cycles; it needs garbage collection or cycle detection to avoid infinite traversal.

Comparison of directory structures

Basis	Single-level	Two-level	Tree-structured (multi-level)
Structure	One directory for all files.	MFD plus one UFD per user.	Root with subdirectories to any depth.
File naming	Names must be unique system-wide.	Unique within a user only.	Unique path names; same name allowed in different directories.
Grouping of files	Not possible.	Only by user.	Logical grouping by subdirectories.
Searching	Slow when many files.	Faster (only the user's UFD).	Efficient using path names and current directory.
Sharing / protection	None / very weak.	Isolation; sharing is difficult.	Sharing via paths; permissions per directory.
Suitability	Very small, single-user systems.	Small multi-user systems.	Modern OS (UNIX, Windows, Linux, macOS).

(ii) Disk Fragmentation and Management Techniques

Fragmentation is the wastage or scattering of disk space so that free space cannot be used efficiently. It arises from the way blocks are allocated to files (Silberschatz Ch. 12 and Ch. 10). It reduces usable capacity and slows down I/O because the disk head has to move more.

Types of fragmentation

Type	Meaning	Cause	Example
Internal fragmentation	Unused space <i>inside</i> an allocated block/cluster.	Files are allocated in fixed-size blocks; the last block is rarely full.	Block = 4 KB, file = 10 KB needs 3 blocks (12 KB): 2 KB wasted.
External fragmentation	Free space exists but is broken into many small non-contiguous holes.	Contiguous allocation with repeated creation/deletion of variable-sized files.	Holes of 2, 3 and 4 blocks cannot store a 7-block contiguous file.
Data (file) fragmentation	The blocks of one file are scattered over the disk.	Non-contiguous (linked/indexed) allocation, file growth, deletion.	A file of 8 blocks is stored in 5 separate places; head seeks increase.

Average internal fragmentation is about **half a block per file**: larger blocks give better transfer rates but more wastage; smaller blocks give less wastage but more overhead and more seeks.

Effects of fragmentation

- Reduced effective disk capacity and inability to create large contiguous files.
- Increased seek time and rotational latency, so lower throughput and slower file access.
- More head movement causes mechanical wear and higher power consumption; higher probability of data-recovery problems.

Management techniques

A. Techniques to prevent / reduce fragmentation

- **Choose a suitable block (cluster) size** - a compromise between internal fragmentation and access speed; some file systems use *tail packing / fragments* for small files.
- **Non-contiguous allocation** - linked (FAT) and indexed (inode) allocation remove external fragmentation because a file can use any free block.
- **Extents** - allocate a contiguous chunk plus links to further chunks (used by ext4, NTFS, XFS): keeps most of the file contiguous while allowing growth.
- **Good placement policies** - first-fit, best-fit or worst-fit for contiguous holes; *delayed allocation* and *pre-allocation* (ext4) so that the final size is known before blocks are chosen.
- **Buddy system** - allocation in power-of-two chunks reduces external fragmentation at the cost of some internal fragmentation.

B. Techniques to remove existing fragmentation

- **Compaction / Defragmentation** - rearranges files so that each file is contiguous and free space is combined into one large hole. Can be done off-line (dump and restore the volume) or on-line by a defragmenter utility (Windows Defragment, e4defrag). Costly in time, so it is scheduled during idle time.
- **Backup and restore** - copy all files to another device, format the volume and copy back; files are then written contiguously.
- **SSDs** - defragmentation is unnecessary (no seek time) and harmful (wears out flash); use *TRIM* and wear levelling instead.

C. Disk management (Silberschatz Sec. 10.5)

Task	Description
Low-level (physical) formatting	Divides the disk into sectors with header, data area and trailer (ECC) so the controller can read and write.
Partitioning	Divides the disk into one or more groups of cylinders, each treated as a separate logical disk.
Logical formatting	Creates the file system on a partition: initial free-space map, empty directory, FCBs (inodes).
Boot block	Bootstrap loader stored at a fixed location; ROM code loads the full bootstrap from the boot block (Windows: MBR + boot partition).
Bad-block management	Sector sparing (forwarding) replaces a bad sector by a spare; sector slipping shifts sectors to free the spare.
Free-space management	Bit vector, linked list, grouping, counting - to find free blocks quickly and reduce fragmentation.
Disk scheduling	FCFS, SSTF, SCAN, C-SCAN, LOOK order requests to reduce seek time and head movement.
Swap-space management	Separate swap partition or file for virtual memory, allocated in large contiguous chunks.

Exam tip - key points to remember

- Single-level: one directory, unique names, no grouping. Multi-level: two-level (MFD/UFD), tree, acyclic graph, general graph.
- Internal fragmentation = waste inside allocated block. External = scattered free holes. Solve by paging-like allocation, extents, compaction/defragmentation.
- Disk management = formatting, partitioning, boot block, bad-block handling.

Q3 (b) [10 Marks, CO1]**What is starvation in SJF? Suggest a method / algorithm to overcome it.****1. Shortest-Job-First (SJF) scheduling**

SJF associates with each process the **length of its next CPU burst**. When the CPU is available it is assigned to the process with the **smallest next CPU burst**; ties are broken by FCFS. SJF is **provably optimal** - it gives the minimum average waiting time for a given set of processes. It has two forms: **non-preemptive** (once given the CPU the process runs to the end of its burst) and **preemptive**, called **Shortest-Remaining-Time-First (SRTF)** (a newly arrived process with a shorter burst than the remaining time of the running process preempts it).

Example (Silberschatz): P1 = 6, P2 = 8, P3 = 7, P4 = 3 (all at time 0). SJF order: P4, P1, P3, P2. Waiting times: 3, 16, 9, 0 → average = 28 / 4 = **7 ms**, whereas FCFS gives 10.25 ms.



Fig. 4 - Non-preemptive SJF for the example above

Difficulty: the length of the next CPU burst is not known in advance. It is **predicted** by exponential averaging: $\tau_{n+1} = \alpha \tau_n + (1 - \alpha) \tau_n$, where τ_n is the length of the n-th actual burst, τ_n the previous prediction and $0 \leq \alpha \leq 1$ (commonly $\alpha = 1/2$).

2. What is starvation?

Starvation (indefinite blocking) is a situation in which a process is ready to run but is **never given the CPU** (or waits for an unbounded time) because other processes are always chosen before it. In SJF the scheduler always prefers the shortest burst, so a process with a **long CPU burst** may wait forever if a steady stream of shorter processes keeps arriving. The scheduler is 'unfair' to long jobs even though the system is busy and productive. Silberschatz notes that a major problem with priority scheduling algorithms (of which SJF is a special case, with priority = 1 / next burst) is indefinite blocking or starvation - a legend says that when the IBM 7094 at MIT was shut down in 1973 they found a low-priority process that had been submitted in 1967 and had not yet been run.

Example demonstrating starvation (non-preemptive SJF)

P1 (BT 3) arrives at 0. Long process L (BT 12) arrives at time 1. Then a short process S_k (BT 3) arrives every 3 units (at 2, 5, 8, 11 ...).

Decision time	Ready queue (burst)	Process chosen	Reason
0	P1 (3)	P1	Only process available.
3	L (12), S1 (3)	S1	3 < 12, so the short job wins.
6	L (12), S2 (3)	S2	S2 arrived at 5, shorter than L.
9	L (12), S3 (3)	S3	S3 arrived at 8, shorter than L.
12, 15, ...	L (12), S4, S5 ...	S4, S5 ...	Every time a new short job is present, L is bypassed - L starves .

Since a short process is always waiting when the CPU becomes free, L never runs. In a system with continuous arrivals the waiting time of L can grow without bound. The same happens in SRTF, even faster because the arriving short jobs also preempt L.

3. Solutions to overcome starvation

Method 1: Aging (the standard solution)

Aging is a technique of **gradually increasing the priority of processes that wait in the system for a long time**. In SJF the 'priority' is the predicted burst, so an aged process has its *effective burst* reduced the longer it waits. Silberschatz's example: if priorities range from 127 (low) to 0 (high), the priority of a waiting process is increased by 1 every 15 minutes. Even a process that enters with priority 127 will reach priority 0 in no more than **32 hours** and will therefore get the CPU.

Aging-based SJF (run each time the CPU is free, and on every clock tick)

```
for every process p in ready queue:
    p.waiting += 1                // time spent waiting so far
    p.effective_burst = p.predicted_burst - K * p.waiting    // K = aging factor
    if p.effective_burst < 1: p.effective_burst = 1
select p with the smallest effective_burst    // ties -> FCFS
dispatch p
```

Effect on the example: with $K = 0.5$, after L has waited 30 units its effective burst is $12 - 15 < 1 \rightarrow 1$, which is smaller than the burst (3) of a new short process, so L is dispatched. Every process is therefore guaranteed to run after a bounded delay.

Method 2: Highest Response Ratio Next (HRRN) - a non-preemptive aging algorithm

HRRN selects the process with the highest **response ratio** = $(W + S) / S$, where W = waiting time and S = expected service (burst) time. A short job has a high ratio at once, but as a long job waits, W grows and its ratio increases until it exceeds that of newly arrived short jobs. Example: L ($S = 20$) after waiting 30 units has ratio $(30 + 20)/20 = 2.5$; a newly arrived short process ($S = 2, W = 2$) has ratio $(2 + 2)/2 = 2.0$. So L is selected.

Other ways to avoid starvation

- **Multilevel feedback queue (MLFQ)** - a process that waits too long in a low-priority queue is *moved to a higher-priority queue* (aging implemented by promotion).
- **Round Robin** - gives every process a time quantum in turn, so every process is served within $(n - 1) \times q$ time units; no starvation, at the cost of higher average waiting time.
- **Guaranteed maximum waiting time / deadline** - if a process waits longer than a threshold, it is scheduled next irrespective of its burst.
- **Fair-share / lottery scheduling** - each process holds tickets, so every process has a non-zero probability of being chosen (Linux CFS uses virtual runtime to ensure fairness).

Summary table

Algorithm	Starvation possible?	Reason / cure
FCFS	No	Served strictly in arrival order (but convoy effect exists).
SJF / SRTF	Yes	Long jobs are postponed by a stream of short jobs; cure: aging.
Priority scheduling	Yes	Low-priority processes are postponed; cure: aging.
Round Robin	No	Each process gets a quantum in every cycle.
HRRN / SJF with aging / MLFQ with promotion	No	Waiting time is taken into account when choosing.

Exam tip - key points to remember

- Starvation = indefinite blocking of a ready process. In SJF, long jobs starve when short jobs keep arriving.
- Standard remedy = **aging**: raise priority (reduce effective burst) with waiting time. Alternatives: HRRN, MLFQ with promotion, Round Robin.

Question 4

10 Marks | Attempt any ONE sub-question in the exam | CO2 / CO1 | Both options are answered below for revision

Q4 (a) [10 Marks, CO2]

Write a detailed note on SVR3 scheduling algorithms and their importance in modern systems.

Note on sources

Silberschatz (9th Ed., Ch. 5) does not have a separate section on UNIX SVR3. This answer uses the description of the *traditional UNIX scheduler* from M. Bach, 'The Design of the UNIX Operating System' (SVR2/SVR3) and W. Stallings, 'Operating Systems' (Traditional UNIX Scheduling), and relates it to the multilevel-feedback-queue, priority and round-robin concepts of Silberschatz Sec. 5.3.

1. Introduction

UNIX System V Release 3 (SVR3) was released by AT&T in 1987 and was one of the most widely used commercial UNIX versions. Its process scheduler is a **time-sharing scheduler** designed to give good response to interactive users while still giving CPU-bound jobs a fair share. It is a **multilevel feedback queue** algorithm that uses **dynamic priorities** and **round-robin scheduling within each priority level**. The priority of a process is recomputed regularly from its *recent CPU usage*, so processes that have used a lot of CPU are pushed down and those that have been waiting or doing I/O move up. The same design (with variations) underlies 4.3BSD and many later UNIX-like systems.

2. Process priorities

- Every process has a numeric **priority**; a **lower number means a higher priority** (the opposite of the intuitive scale).
- Priorities are divided into two groups: **kernel-mode (sleep) priorities** and **user-mode priorities**. All kernel priorities are numerically lower (better) than any user priority.
- **Kernel-mode / sleep priorities** are *fixed* and depend on the reason for which a process is sleeping inside the kernel. In decreasing order of priority the bands are: **swapper** → **block-I/O device control** (waiting for disk I/O, buffers, inodes) → **file manipulation** → **character-I/O device control** (terminal input/output) → **user processes**. A process sleeping for disk I/O gets a high priority because it holds scarce kernel resources (buffers, inodes) that others may need - it should finish quickly.
- The boundary between the two kinds of sleep is called the **threshold priority**: a process sleeping at a high priority (e.g. waiting for disk I/O) is *not* woken by signals, whereas one sleeping at a lower priority (e.g. waiting for terminal input) can be interrupted by a signal.
- **User-mode priorities** are *dynamic*: they are computed by the scheduler when the process returns to user mode and are periodically recalculated.

3. Algorithm - how the scheduler works

Each process's process-table entry holds the fields **p_pri** (current priority), **p_cpu** (recent CPU-usage count) and **p_nice** (user-controlled *nice* value; default is neutral, a positive nice value lowers the priority of a background job).

1. **Clock tick**: the clock interrupts the system many times a second (typically 50-100 Hz). At each tick the kernel increments p_cpu of the running process.
2. **Every second (decay)**: the clock handler visits every process and applies the **decay function**: $CPU_j(i) = CPU_j(i-1) / 2$. Old CPU usage thus counts less and less (exponentially).
3. **Priority recalculation**: for each process in user mode the priority is recomputed: $P_j(i) = Base_j + CPU_j(i)/2 + nice_j$, where $Base_j$ is the base priority of the user band. Then the process is placed in the run queue of that priority.

4. **Selection:** the scheduler (*swtch()*) picks the **highest-priority runnable process** that is in memory. Processes of the same priority are served **round-robin** (FIFO order in the queue).
5. **Time slice:** a running process is preempted when its time quantum (about 1 second) expires, or earlier if it blocks. Because of recalculation, a CPU-bound process's priority number grows (priority falls) as it runs, so another process takes over.
6. **Sleep and wakeup:** when a process blocks on an event it is given the sleep priority of that event. On *wakeup()* it becomes runnable at that high kernel priority, runs to complete the kernel work and, on return to user mode, its priority is reset to the user priority computed by the formula.
7. **Non-preemptive kernel:** a process running in kernel mode is *not preempted* by another process (only interrupts can interrupt it); a context switch occurs only when it sleeps, exits, or returns to user mode. This protects kernel data structures without locks but makes real-time response poor.

Worked example (three CPU-bound processes A, B, C)

Assume Base = 60, the clock ticks 60 times per second (so a process running for one full second gets CPU count 60), and the priorities are recomputed each second using integer division. Each cell shows **CPU count / priority** at the *start* of the second (after recalculation).

Time (s)	Process A	Process B	Process C	Process chosen (lowest number)
0	0 / 60	0 / 60	0 / 60	A (tie, first in queue)
1	30 / 75	0 / 60	0 / 60	B
2	15 / 67	30 / 75	0 / 60	C
3	7 / 63	15 / 67	30 / 75	A
4	33 / 76	7 / 63	15 / 67	B

The example shows that the processes share the CPU in a fair, rotating way: the process that has just run gets a worse priority, and the decay gives the ones that waited a better priority. An I/O-bound process would show a very small CPU count, so it keeps a good priority and gets the CPU promptly after each I/O completion.

4. Features, advantages and limitations

Feature / Advantage	Limitation
Favors interactive and I/O-bound processes; they use little CPU and keep good priority.	Kernel is non-preemptive: a high-priority process may wait while a low-priority one is inside the kernel (poor real-time behaviour).
Decay (aging) prevents starvation - old CPU usage is forgotten every second.	Periodic recalculation of all processes every second costs $O(n)$ time - does not scale to thousands of processes.
Round robin among equal priorities gives fairness; simple to implement with per-priority queues.	No fairness between users (a user with many processes gets more CPU); no guaranteed response or deadlines.
The <i>nice</i> value gives users/administrators simple control over background jobs.	Priorities can be difficult to tune; no support for multiprocessor scheduling or processor affinity.
Sleep priorities let the kernel finish critical work (disk, buffers) quickly.	Possible priority inversion when a low-priority process holds a resource needed by a high-priority one.

5. Evolution: from SVR3 to SVR4 and modern systems

Aspect	SVR3 (traditional)	SVR4 and later	Modern Linux (CFS) / Windows
Scheduling classes	Single time-sharing class.	Real-time, system (kernel) and time-shared classes with about 160 priority levels.	Linux: fair class, real-time (FIFO/RR), deadline; Windows: 32 priority levels.
Priority change	Recalculated each second from CPU usage.	Table-driven (dispatch table) - administrator can tune.	Virtual runtime (CFS); dynamic boosts after I/O in Windows.
Kernel preemption	None.	Preemption points in the kernel.	Fully preemptible kernel options.
Time quantum	About 1 s fixed.	Configurable per priority level.	Proportional / dynamic slice.
Multiprocessor	Not supported.	Basic support.	Per-CPU run queues, load balancing, affinity.

6. Importance in modern systems

- Foundation of feedback scheduling:** the idea that a process's priority depends on its past behaviour is the basis of MLFQ (Silberschatz Sec. 5.3.6) and is used in Solaris, FreeBSD, macOS and older Linux (O(1)) schedulers.
- Aging and decay:** the halving of CPU usage is an early form of aging, which is still the standard cure for starvation in priority schedulers.
- Interactive responsiveness:** it introduced the strategy of rewarding I/O-bound (interactive) tasks and penalising CPU hogs - now used in all desktop and mobile operating systems.
- The nice value:** user-controlled priority adjustment is still present in Linux (nice, renice) and all UNIX-like systems.
- Separation of kernel and user priorities:** the concept that kernel work has higher priority than user work carries into modern real-time and preemptible-kernel designs.
- Educational value:** it shows the trade-offs between simplicity, fairness, response time and throughput that every scheduler design must balance.

Exam tip - key points to remember

- SVR3 = time-sharing, multilevel feedback queues, round robin within a priority level, lower number = higher priority.
- Formulae: $CPU(i) = CPU(i-1)/2$ and $Priority = Base + CPU(i)/2 + nice$, recalculated once every second.
- Kernel priorities (swapper, disk I/O, file, tty) are fixed; user priorities are dynamic. Kernel is non-preemptive.

Q4 (b) [10 Marks, CO1]

Using Round Robin scheduling with time quantum = 2 ms, for processes P1 (AT 0, BT 5), P2 (AT 1, BT 4), P3 (AT 2, BT 6), P4 (AT 3, BT 3): prepare the Gantt chart and calculate WT, TAT and Response Time for each process.

1. Round Robin (RR) scheduling - concept

Round Robin is designed especially for **time-sharing systems**. It is like FCFS but with **preemption**: a small unit of time called the **time quantum (time slice)** is defined (here $q = 2$ ms). The ready queue is treated as a circular FIFO queue. The scheduler takes the first process, sets a timer for one quantum and dispatches it. Then either (i) the process finishes before the quantum ends and releases the CPU voluntarily, or (ii) the quantum expires, the timer interrupts, a context switch occurs and the process is put at the **tail** of the ready queue. A newly arriving process is added at the tail too. (Silberschatz Sec. 5.3.4)

Convention used: if a new process arrives at the same instant at which the running process is preempted, the *new arrival is placed in the queue first*, followed by the preempted process. Context-switch time is ignored.

Formulae

- **TAT** = Completion Time (CT) - Arrival Time (AT)
- **WT** = TAT - Burst Time (BT)
- **Response Time (RT)** = time of *first* allocation of the CPU - Arrival Time

2. Given data ($q = 2$ ms)

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	4
P3	2	6
P4	3	3

3. Step-by-step execution (ready-queue trace)

Time slice	Process runs	Remaining after slice	Ready queue after the slice (head to tail)	Remark
0 - 2	P1	$5 - 2 = 3$	P2, P3, P1	P2 ($t=1$) and P3 ($t=2$) arrived, queued before P1
2 - 4	P2	$4 - 2 = 2$	P3, P1, P4, P2	P4 arrived at $t=3$
4 - 6	P3	$6 - 2 = 4$	P1, P4, P2, P3	
6 - 8	P1	$3 - 2 = 1$	P4, P2, P3, P1	
8 - 10	P4	$3 - 2 = 1$	P2, P3, P1, P4	First run of P4
10 - 12	P2	$2 - 2 = 0$	P3, P1, P4	P2 completes at 12
12 - 14	P3	$4 - 2 = 2$	P1, P4, P3	
14 - 15	P1	$1 - 1 = 0$	P4, P3	P1 completes at 15 (needs only 1 ms)
15 - 16	P4	$1 - 1 = 0$	P3	P4 completes at 16
16 - 18	P3	$2 - 2 = 0$	empty	P3 completes at 18

4. Gantt chart

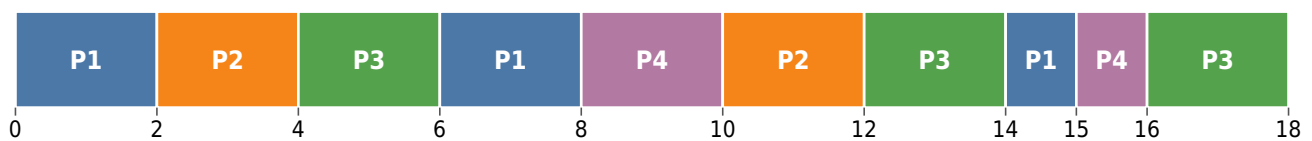


Fig. 5 - Gantt chart for Round Robin ($q = 2$ ms); total time = $5 + 4 + 6 + 3 = 18$ ms

5. Calculation of CT, TAT, WT and RT

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT	First CPU	RT = First - AT
P1	0	5	15	$15 - 0 = 15$	$15 - 5 = 10$	0	$0 - 0 = 0$
P2	1	4	12	$12 - 1 = 11$	$11 - 4 = 7$	2	$2 - 1 = 1$
P3	2	6	18	$18 - 2 = 16$	$16 - 6 = 10$	4	$4 - 2 = 2$
P4	3	3	16	$16 - 3 = 13$	$13 - 3 = 10$	8	$8 - 3 = 5$
Total		18		55	37		8
Average				13.75	9.25		2.0

Average TAT = $55 / 4 = 13.75$ ms **Average WT** = $37 / 4 = 9.25$ ms **Average RT** = $8 / 4 = 2.0$ ms

6. Other observations

- **CPU utilisation** = 100% (no idle time), **throughput** = $4 / 18 = 0.22$ processes per ms.
- **Number of context switches** = 9 (10 slices, not counting the initial dispatch).
- Response time is small for every process (maximum 5 ms) - the main advantage of RR - but the average waiting time (9.25 ms) is higher than that of SJF because of extra waiting between slices.
- If the quantum were very large, RR would behave like FCFS; if very small, context-switch overhead would dominate. A good rule: about 80% of CPU bursts should be shorter than the quantum.

Verification

Total WT = total TAT - total BT = $55 - 18 = 37$ ✓. Last completion time = 18 = sum of all burst times (no idle time)
✓

Question 5

10 Marks | Attempt any ONE sub-question in the exam | CO1 | Both options are answered below for revision

Q5 (a) [10 Marks, CO1]

Explain with examples how CPU scheduling affects system performance.

1. Introduction

CPU scheduling is the basis of multiprogrammed operating systems. By switching the CPU among processes, the OS makes the computer more productive. Whenever the CPU becomes idle, the **short-term (CPU) scheduler** selects a process from the ready queue and the **dispatcher** gives it control of the CPU (switching context, switching to user mode, jumping to the proper location). The time taken by the dispatcher to stop one process and start another is called **dispatch latency**. Scheduling decisions take place when a process (1) switches from running to waiting, (2) switches from running to ready, (3) switches from waiting to ready, or (4) terminates. If scheduling occurs only at 1 and 4 it is **non-preemptive**; otherwise it is **preemptive** (Silberschatz Sec. 5.1).

Process execution is a cycle of **CPU burst** and **I/O burst**. An *I/O-bound* program has many short CPU bursts; a *CPU-bound* program has a few long CPU bursts. How well the scheduler mixes these two kinds of processes directly decides the performance of the entire system.

2. Performance criteria affected by scheduling

Criterion	Meaning	Goal
CPU utilisation	Percentage of time the CPU is busy (should range from 40% for a lightly loaded to 90% for a heavily loaded system).	Maximise
Throughput	Number of processes completed per unit time.	Maximise
Turnaround time	Time from submission to completion (waiting for memory + ready queue + CPU + I/O).	Minimise
Waiting time	Total time a process spends waiting in the ready queue.	Minimise
Response time	Time from submission of a request until the first response is produced - important for interactive systems.	Minimise
Fairness (and variance)	Each process gets a fair share; predictable, low variance in response time; no starvation.	Maximise

These goals conflict: e.g. maximising throughput (by running short jobs first) may hurt fairness for long jobs; giving quick response (small quantum) increases context-switch overhead and reduces throughput. The choice of scheduling algorithm is therefore a trade-off.

3. Effect of scheduling on performance - with examples

Example 1: Order of execution changes the average waiting time (FCFS and the convoy effect)

Three processes arrive at time 0 with CPU bursts $P1 = 24$ ms, $P2 = 3$ ms, $P3 = 3$ ms.

Order (FCFS)	Gantt chart	Waiting times	Average WT
P1, P2, P3	P1 (0-24) P2 (24-27) P3 (27-30)	$P1 = 0, P2 = 24, P3 = 27$	$(0 + 24 + 27) / 3 = 17$ ms
P2, P3, P1	P2 (0-3) P3 (3-6) P1 (6-30)	$P2 = 0, P3 = 3, P1 = 6$	$(0 + 3 + 6) / 3 = 3$ ms

The same processes give a 5-times better average waiting time if the short ones run first. When one long CPU-bound process holds the CPU, all short (I/O-bound) processes wait behind it - the **convoy effect** - which results in **lower CPU and device utilisation**, because the I/O devices sit idle while the short processes wait for the CPU.

Example 2: SJF minimises average waiting time but may hurt fairness

$P1 = 6, P2 = 8, P3 = 7, P4 = 3$ ms (all arrive at 0). FCFS gives average WT = $(0 + 6 + 14 + 21)/4 = 10.25$ ms; SJF (order $P4, P1, P3, P2$) gives $(3 + 16 + 9 + 0)/4 = 7$ ms. SJF has the best average waiting time and highest throughput of short jobs, but long jobs may starve ($Q3$ (b)), and the burst length must be predicted.

Example 3: Time quantum in Round Robin decides response time and overhead

With $P1 = 24, P2 = 3, P3 = 3$ and quantum 4 ms the order is $P1$ (0-4), $P2$ (4-7), $P3$ (7-10), $P1$ (10-14), ... $P1$ (26-30). Waiting times: $P1 = 6, P2 = 4, P3 = 7$, average = **5.66 ms** (compare 17 ms for FCFS) and each process gets its first response within 7 ms. But the quantum must also be chosen against the context-switch time:

Quantum q	Effect on a process needing 10 ms	Effect on the system
q = 12 ms (large)	0 context switches; finishes in one slice.	Behaves like FCFS: poor response for short jobs.
q = 6 ms	1 context switch.	Balanced.
q = 1 ms (very small)	9 context switches.	Response is excellent but overhead is large - processor sharing.

Overhead calculation: if $q = 4$ ms and one context switch takes 1 ms, useful work = $4 / (4 + 1) = 80\%$, so **20% of the CPU is wasted**. With $q = 20$ ms the waste falls to about 4.8%, but response time increases. A good rule: about 80% of CPU bursts should be shorter than the quantum.

Example 4: I/O-bound vs CPU-bound mix keeps devices busy

Let a CPU-bound job need 100 ms of CPU and an I/O-bound job use 1 ms of CPU followed by a 10 ms disk operation, repeatedly. If the scheduler lets the CPU-bound job run first without preemption, the disk stays **idle for 100 ms**, and both CPU and disk are under-used in the long run. If the I/O-bound job is given priority, it starts its disk operation after 1 ms and then the CPU-bound job runs while the disk works - **CPU and I/O device are busy together**, giving higher utilisation and throughput. This is why multilevel feedback queues (and the UNIX scheduler in Q4 (a)) favour I/O-bound processes.

Example 5: Priority scheduling and real-time systems

A media player or an airbag controller must be dispatched within a deadline. Priority (or real-time) scheduling gives these tasks the CPU first, so they meet their deadlines; but low-priority batch jobs may **starve** unless aging is used. A low-priority process holding a lock needed by a high-priority process causes **priority inversion**, cured by *priority inheritance*.

Other effects of scheduling

- **Multilevel queue / feedback queue** - separate queues for foreground (interactive, RR) and background (batch, FCFS) jobs give good response to users and good throughput to batch jobs.
- **Multiprocessor scheduling** - *load balancing* keeps all CPUs busy, while *processor affinity* keeps a process on one CPU to reuse its cache. Poor choices reduce cache hits and utilisation.
- **Modern schedulers** - Linux CFS gives each task a fair share of CPU using virtual runtime; Windows uses a priority-based preemptive scheduler with priority boosts for interactive threads.

4. Which scheduler for which system?

System type	Main goal	Suitable scheduling
Batch (payroll, scientific jobs)	Maximum throughput, low turnaround.	FCFS, SJF, priority with aging.
Interactive / time-sharing	Low response time, fairness.	Round Robin, multilevel feedback queue.
Real-time (embedded, multimedia)	Meet deadlines.	Priority-based preemptive, rate-monotonic, EDF.
Multiprocessor / servers	Load balancing, cache affinity.	Per-CPU queues with balancing (Linux CFS).

Exam tip - key points to remember

- Performance metrics: CPU utilisation, throughput, turnaround, waiting, response time. A good scheduler maximises the first two and minimises the rest.
- Always support with a numeric example: FCFS 17 ms vs 3 ms by order; SJF 7 ms vs FCFS 10.25 ms; RR quantum 4 ms gives 5.66 ms.
- Too large a quantum = FCFS; too small = context-switch overhead.

Q5 (b) [10 Marks, CO1]

Compare Round Robin with FCFS and SJF in terms of: Fairness, Throughput, Average waiting time.

1. The three algorithms in brief

Algorithm	Selection rule	Preemptive?	Ready-queue data structure
FCFS	The process that arrived first gets the CPU first.	No	FIFO queue
SJF	The process with the smallest next CPU burst gets the CPU (SRTF is the preemptive form).	No (SJF) / Yes (SRTF)	Priority queue ordered by burst length
Round Robin	Each process gets a fixed time quantum q in circular order.	Yes (by timer)	Circular FIFO queue

2. Comparison table

Basis	FCFS	SJF	Round Robin
Fairness	Fair in arrival order, but unfair to short jobs behind a long one (convoy effect).	Unfair to long jobs; they may starve if short jobs keep arriving.	Most fair: every process gets equal CPU share in turn; no starvation.
Starvation	No.	Yes (cure: aging).	No.
Throughput	Low to medium; suffers from convoy effect.	Highest: short jobs finish quickly, so more jobs complete per unit time.	Medium; reduced by context-switch overhead.
Average waiting time	Often long; depends on arrival order.	Minimum (provably optimal) for a given set.	Usually longest of the three, because of repeated waiting between slices.
Response time	Poor for short jobs.	Good for short jobs, poor for long jobs.	Best: small and predictable.
Turnaround time	Depends on order.	Lowest average.	High average, especially for equal-length jobs.
Preemption	Non-preemptive.	Non-preemptive or preemptive (SRTF).	Preemptive.
Context-switch overhead	Lowest (only at completion).	Low (SRTF: more).	Highest; depends on quantum.
Implementation	Very easy (FIFO).	Hard: needs the next burst length, which is predicted.	Easy; needs a timer and choice of q .
Suitable for	Batch systems.	Batch systems with known run times.	Time-sharing / interactive systems.

3. Discussion of each criterion

(i) Fairness

Round Robin gives every process the same quantum in turn, so with n processes each process waits at most $(n - 1) \times q$ before it gets the CPU again - a guaranteed bound, hence no starvation and the highest fairness. **FCFS** is fair with respect to arrival time only; a short job may wait a long time behind a CPU-hog. **SJF** is the least fair because it always favours short jobs; long processes can be postponed indefinitely.

(ii) Throughput

Throughput is the number of processes completed per unit time. **SJF** gives the highest throughput since it completes the maximum number of short jobs early. **FCFS** gives lower throughput when a long job is at the head. **RR** finishes short jobs quickly only if q is larger than their bursts; the context-switch overhead and the delay in completing every job reduce its throughput slightly. For a large q , RR behaves like FCFS; for a very small q the overhead becomes large.

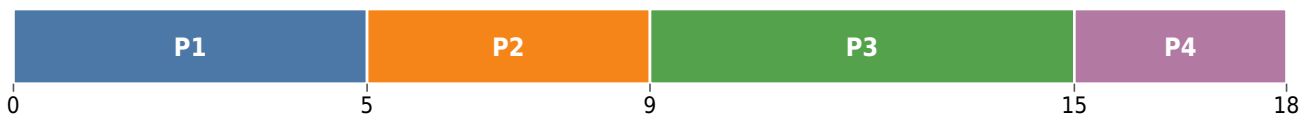
(iii) Average waiting time

SJF has the minimum average waiting time because moving a short job before a long one reduces the waiting of the short job by more than it increases that of the long job. **FCFS** average waiting time is often long. **RR** average waiting time is often long as well, because a process repeatedly re-enters the queue; its advantage lies in the response time, not in the waiting time.

4. Numerical demonstration (data from Q4 (b))

Processes: P1 (AT 0, BT 5), P2 (AT 1, BT 4), P3 (AT 2, BT 6), P4 (AT 3, BT 3). RR uses $q = 2$ (worked out in Q4 (b)); SJF is non-preemptive.

FCFS Gantt chart



SJF (non-preemptive) Gantt chart - at $t = 5$ the ready queue holds P2 (4), P3 (6), P4 (3), so P4 is chosen, then P2, then P3.



Measure	FCFS	SJF (non-preemptive)	Round Robin ($q = 2$)
Completion times P1 / P2 / P3 / P4	5 / 9 / 15 / 18	5 / 12 / 18 / 8	15 / 12 / 18 / 16
Waiting times P1 / P2 / P3 / P4	0 / 4 / 7 / 12	0 / 7 / 10 / 2	10 / 7 / 10 / 10
Average waiting time	$23 / 4 = 5.75$	$19 / 4 = 4.75$	$37 / 4 = 9.25$
Average turnaround time	$41 / 4 = 10.25$	$37 / 4 = 9.25$	$55 / 4 = 13.75$
Average response time	5.75	4.75	2.0
Jobs completed by $t = 12$ (throughput view)	2 (P1, P2)	3 (P1, P4, P2)	1 (P2)
Number of context switches	3	3	9

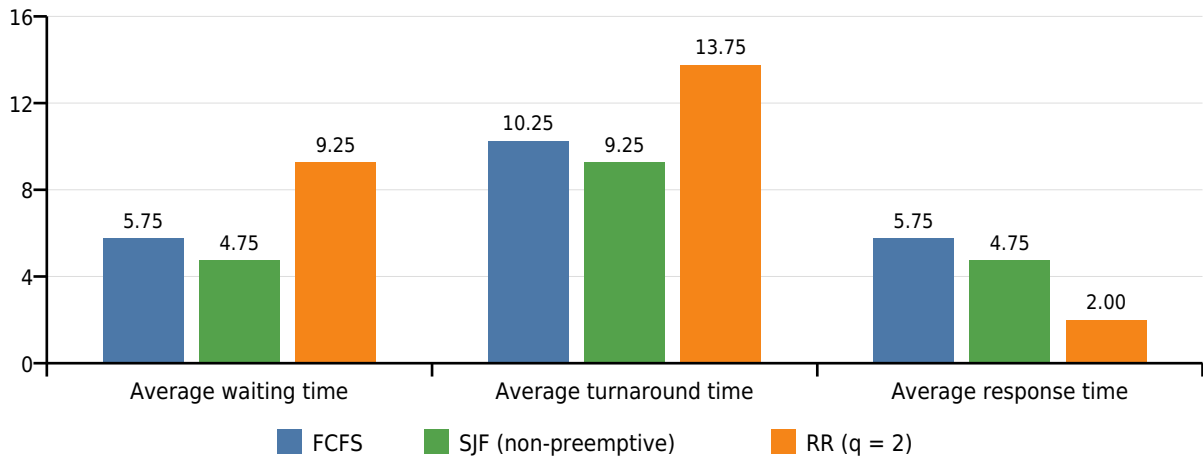


Fig. 6 - Average waiting, turnaround and response times for the Q4 (b) data

Conclusion of the demonstration: SJF has the lowest average waiting/turnaround time and completes the most jobs early; RR has by far the best average response time (2.0 ms) and gives every process regular service, but pays with the highest average waiting time and most context switches; FCFS lies in between and is simplest.

5. Final conclusion

No algorithm is best for all situations: choose **FCFS** for simple batch systems, **SJF** when run times are known/predictable and minimum waiting time is the goal (with aging to avoid starvation), and **Round Robin** for time-sharing systems where fairness and response time matter most. Real systems combine them, e.g. a multilevel feedback queue with RR at the higher levels and FCFS at the lowest level.

Exam tip - key points to remember

- Fairness: $RR > FCFS > SJF$. Throughput: $SJF > FCFS \approx RR$. Average waiting time: $SJF \text{ (best)} < FCFS < RR$ (usually). Response time: RR best.
- RR: waiting bound $(n - 1) \times q$; depends on q ; too large = FCFS, too small = overhead.

Quick Revision Sheet

One-page summary of formulas, definitions and practice questions for the whole paper

1. CPU-scheduling formulas

Term	Formula / meaning
Turnaround time (TAT)	Completion time - Arrival time
Waiting time (WT)	TAT - Burst time
Response time (RT)	Time of first CPU allocation - Arrival time
Throughput	Number of processes completed / total time
CPU utilisation	(Busy time / total time) x 100
Response ratio (HRRN)	(Waiting time + Service time) / Service time
SJF burst prediction	$\tau(n+1) = \alpha \times t(n) + (1 - \alpha) \times \tau(n)$
RR waiting bound	$(n - 1) \times q$ for n processes
RR overhead	Efficiency = $q / (q + \text{context-switch time})$
UNIX SVR3 priority	$\text{CPU}(i) = \text{CPU}(i-1)/2$; Priority = Base + $\text{CPU}(i)/2$ + nice

2. Scheduling algorithms at a glance

Algorithm	Type	Advantage	Weakness
FCFS	Non-preemptive	Simple, no starvation.	Convoy effect, long average WT.
SJF / SRTF	Non-pre. / preemptive	Minimum average WT.	Starvation; burst must be predicted.
Priority	Both	Handles important jobs first.	Starvation, priority inversion (cure: aging).
Round Robin	Preemptive	Fair, best response time.	High overhead, poor average WT.
Multilevel queue	Preemptive	Separate treatment of job classes.	Inflexible, starvation of lower queues.
Multilevel feedback	Preemptive	Adaptive; aging by promotion.	Complex to tune.

3. Key definitions

Topic	Remember
Kernel / Shell	Kernel: privileged core managing resources. Shell: user-mode command interpreter that uses system calls.
File	Named collection of related information on secondary storage; attributes: name, id, type, location, size, protection, time.
File organization	Pile, sequential, indexed sequential, indexed, direct (hashed).
Access methods	Sequential (read next), direct (read n), indexed (index then record).
Allocation methods	Contiguous (fast, external fragmentation), linked (no direct access), indexed (index block / inode).
Directory structures	Single-level, two-level, tree, acyclic graph, general graph.
Fragmentation	Internal (inside block), external (scattered holes); cure: compaction, extents, non-contiguous allocation.
Protection	Domain = set of <object, rights>; access matrix rows = domains, columns = objects; ACL vs capability list.
Access control	Authenticate, then authorise: ACL, owner/group/universe (9 bits), RBAC; principle of least privilege.
Starvation	Indefinite waiting of a ready process; solved by aging.

4. Practice questions from the same syllabus

1. Explain the structure of an operating system: monolithic, layered, microkernel and modular approaches.
2. What are system calls? Explain their types with examples.
3. Compare contiguous, linked and indexed allocation methods. Explain the UNIX inode structure.
4. Explain free-space management techniques (bit vector, linked list, grouping, counting).
5. Explain the access matrix and its implementation. How are rights revoked?
6. Solve: SJF (preemptive) and priority scheduling for the data given in Q2 (b) and Q4 (b). Compare with FCFS and RR.
7. What is the multilevel feedback queue scheduler? Explain how it avoids starvation.
8. Explain disk-scheduling algorithms: FCFS, SSTF, SCAN, C-SCAN and LOOK.
9. Differentiate between preemptive and non-preemptive scheduling. What is dispatch latency?
10. Explain thread scheduling and multiprocessor scheduling (load balancing, processor affinity).